FINAL PROJECT REVIEW

ASSQP
ADAPTIVE SOFTWARE SECURITY AND QUALITY PLATFORM

AI-assisted software quality, security and risk assurance across the project lifecycle

FACULTY / GUIDE

Dr. J. Kavitha Reddy

TEAM MEMBERS

2420030540 Allipuram Akhil
2420030546 P. Ruthvik Reddy
2420030579 S. Akshay Veer
2420030141 B. Akhil Varma

PROJECT OVERVIEW

One Workspace for Software Assurance

ASSQP consolidates project-scoped quality metrics, security findings, code churn, test history, risk prediction and CI/CD activity into one dashboard. A deterministic rule-based engine remains available when optional tools or the ML service are unavailable.

ANALYSE

Inspect uploaded source files for complexity, duplication, code smells and maintainability.

ASSESS

Combine vulnerability evidence, git activity, test history and a 10-feature ML prediction into a hybrid risk level.

IMPROVE

Prioritize tests, track pipeline runs, retain prediction history and generate project-level JSON reports.

PROJECT DEFINITION

Why Unified Assurance Is Needed

CHALLENGES

- Coverage, scanner findings, test results and CI logs live in separate tools.
- Teams lack one component-level view of quality and security risk.
- Manual triage makes it difficult to decide what to fix or test first.
- External scanners and ML services may be unavailable or unconfigured.

ASSQP RESPONSE

- Consolidate evidence into a project-scoped dashboard.
- Use rule-based quality and security analysis as the baseline.
- Enrich results with SonarQube, Trivy, ZAP, JaCoCo and git history.
- Combine deterministic risk scoring with ML prediction.
- Feed risk and test outcomes into adaptive test prioritization.

PROJECT DIRECTION

Project Objectives

01 Unify project evidence

Bring metrics, findings, tests and pipeline runs into one view.

02 Automate static analysis

Measure complexity, duplication, code smells and maintainability.

03 Integrate security tools

Connect JaCoCo, SonarQube, Trivy and OWASP ZAP.

04 Assess component risk

Combine probability × impact scoring with ML risk prediction.

05 Adapt test execution

Rank test cases using risk and historical failure signals.

06 Support traceability

Retain git churn, reports, model metadata and prediction history.

PROPOSED METHODOLOGY

Continuous Assurance Lifecycle

A project is registered, source and reports are collected, analysis is executed, risk is assessed, tests are prioritized and the resulting evidence is retained for the next development cycle.

REGISTER

Create project and upload source files.

ANALYSE

Run quality and rule-based security checks.

ENRICH

Import scanner, coverage and git-history signals.

ASSESS

Calculate hybrid component risk and test priority.

FEEDBACK

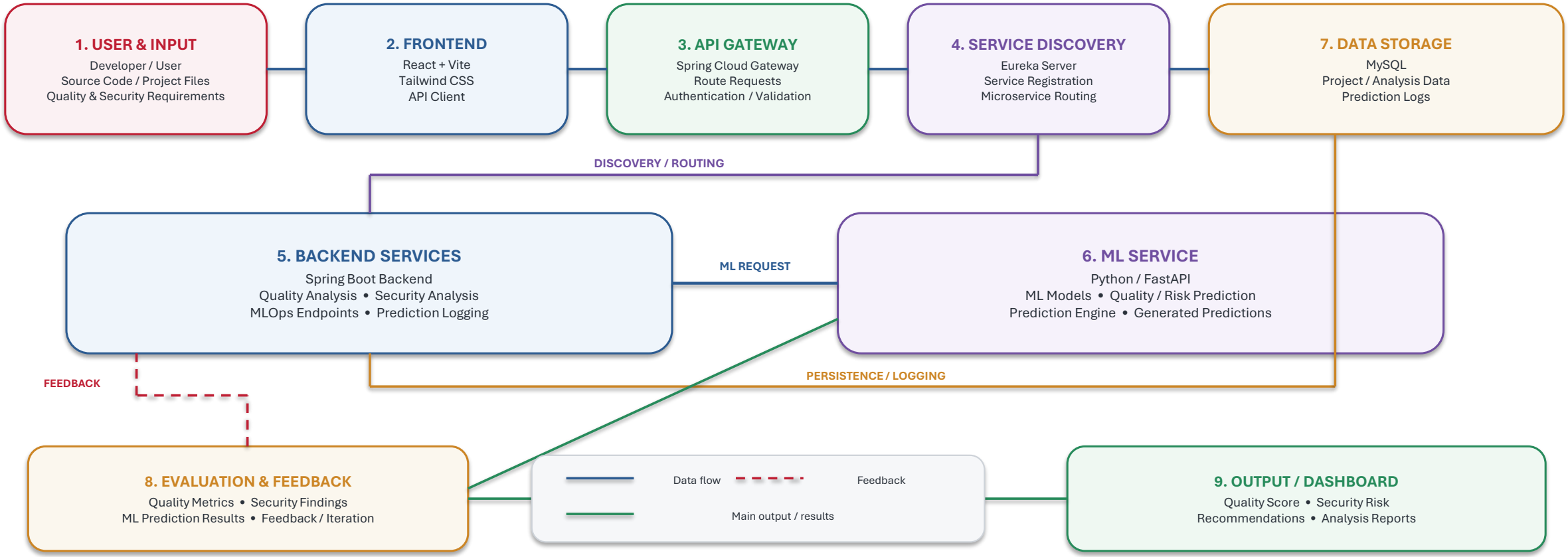
Store outcomes, reports and pipeline history.

SYSTEM ARCHITECTURE

System Design Architecture (PDF View)

SYSTEM ARCHITECTURE

Architecture reproduced from the approved Project System Design submission



ARCHITECTURE DETAIL

Implemented Runtime Architecture

FRONTEND	React 18, TypeScript, Vite, TanStack Query, React Router and Tailwind CSS.
API + SECURITY	Spring Boot 3.3.4 / Java 21, REST controllers, stateless JWT and role-based access.
QUALITY + SECURITY	Rule-based analyzers plus optional SonarQube, Trivy and OWASP ZAP integrations.
TESTING + ADAPTATION	Test cases, execution history and rule-based prioritization using risk and failure-rate signals.
ML SERVICE	Standalone FastAPI service with scikit-learn RandomForestClassifier.
PERSISTENCE	MySQL 8 with 17 JPA entities; Hibernate manages schema updates in the dev profile.
DEVSECOPS	Docker Compose and GitHub Actions pipeline for build, tests, coverage and scanning.

COMPONENT ARCHITECTURE

Platform Modules and Capabilities

Auth & Projects

JWT registration/login, project CRUD, ownership checks and DEVELOPER / ADMIN roles.

Quality Analysis

Complexity, duplication, code smells and maintainability index from uploaded source.

Security Analysis

Rule-based findings plus imported/live scanner results in one vulnerability view.

Coverage + Scanners

JaCoCo XML, SonarQube Web API, Trivy CLI/report import and ZAP baseline/report import.

Git Code Churn

JGit reads local commit count, changed lines and recency-weighted activity.

Hybrid Risk Engine

Rule-based probability × impact plus ML class; final level takes the more severe result.

Adaptive Testing

Prioritizes test execution from risk and historical failure-rate signals.

MLOps + Reports

Model info, metrics, retraining, project-scoped prediction history and JSON reports.

END-TO-END WORKFLOW

From Project Upload to Actionable Results

1 Authenticate
JWT access and refresh tokens

2 Create project
Project-scoped workspace

3 Upload source
Source file metadata

4 Analyse quality
Complexity, duplication, smells

5 Analyse security
Rule-based findings

6 Enrich evidence
JaCoCo / SonarQube / Trivy / ZAP / git

7 Assess risk
Component-level hybrid score

8 Prioritize tests
Risk + failure-rate signals

9 Record execution
PASSED / FAILED / other states

10 Review dashboard
Metrics, findings and trends

11 Generate report
Stored JSON report

12 Repeat cycle
New analysis after code changes

The dashboard combines current project state with historical coverage, analysis, execution and ML prediction records.

ANALYSIS LAYER

Quality, Security and Engineering Signals

QUALITY ANALYSIS

- Cyclomatic complexity
- Duplication and code smells
- Maintainability index
- Uploaded-source analysis

SECURITY ANALYSIS

- Rule-based vulnerability detection
- Findings mapped to affected files
- Severity and source tracking
- Combined vulnerability table

EXTERNAL INTEGRATIONS

- SonarQube Web API
- Trivy CLI or JSON import
- OWASP ZAP baseline or JSON import
- Graceful unavailable state

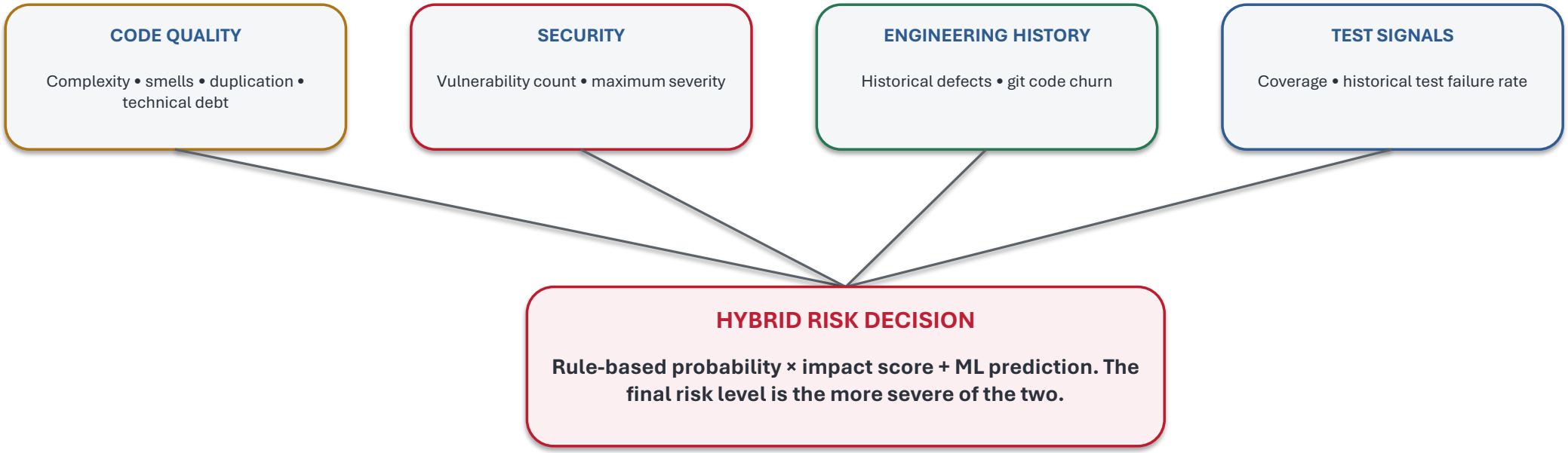
COVERAGE + TESTING

- JaCoCo XML import and trends
- Test case and execution records
- Pass / fail / other outcomes
- Historical failure-rate signals

Live integrations require the corresponding service, credentials or local CLI. Report import remains available where supported.

RISK ASSESSMENT

Hybrid Component Risk Assessment



The ML service predicts LOW / MEDIUM / HIGH / CRITICAL from 10 features. If it is unavailable, rule-based risk scoring remains active.

ADAPTIVE TESTING

Adaptive Test Prioritization

The adaptive engine uses current component risk and historical test outcomes to recommend a test execution order. Each execution cycle contributes feedback for later prioritization.

COLLECT SIGNALS

Component risk, failure rate and test history.

CALCULATE PRIORITY

Apply deterministic rule-based prioritization.

RECOMMEND ORDER

Return an ordered list of test cases.

RECORD OUTCOMES

Capture PASSED / FAILED / other execution states.

FEEDBACK LOOP

Use updated history in the next analysis cycle.

Adaptive prioritization is rule-based; ML supports component risk prediction rather than replacing the test-priority engine.

DEVSECOPS PIPELINE

DevSecOps and Pipeline Visibility

PIPELINE STAGES

1. Checkout repository
2. Build Spring Boot backend
3. Run JUnit tests with JaCoCo
4. Optionally run SonarQube analysis
5. Run Trivy scan stage
6. Record pipeline event through webhook

CONDITIONAL TOOLS

- GitHub Actions workflow
- Pipeline webhook records build/test/coverage/Sonar/Trivy stages
- CI/CD records are project-scoped
- Optional scanners degrade gracefully when unavailable

IMPORTANT CONFIGURATION

Security controls include JWT authentication, BCrypt password storage, role-based authorization and project ownership checks. ZAP live scans are restricted to an explicit approved-target allow-list.

Secrets are read from environment variables; the root .env file is gitignored and must not be committed.

DATA + MLOPS

ML Service and MLOps Monitoring

MYSQL DATA

- Standalone FastAPI microservice
- RandomForestClassifier
- 10 engineered input features
- LOW / MEDIUM / HIGH / CRITICAL output
- Model info and evaluation metrics
- Retraining endpoint
- Service-level prediction log

MLOPS CAPABILITIES

- Backend project-scoped MlPredictionLog
- Model version and training metadata
- Accuracy / precision / recall / F1 endpoints
- Retrain proxy and model hot reload
- Prediction history for each project/component

Dataset disclosure: current training data is synthetic/sample data generated from a documented heuristic. It is not real historical project data.

USER INTERFACE

One Dashboard for Project Assurance

DASHBOARD VIEWS

- Project overview and quality score
- Quality analysis history and trends
- Security findings and severity
- Coverage reports and trends
- Test cases and execution history
- Component risk and ML predictions
- Git churn metrics
- CI/CD pipeline activity

REPORTING + API

- Project-scoped JSON reports
- Swagger UI for 96 API endpoints
- FastAPI interactive documentation
- Recharts visualizations
- Notifications and analysis history

Frontend: React 18, TypeScript, Vite, TanStack Query, React Router, Tailwind CSS and Axios.

IMPLEMENTATION STACK

Technology Stack

FRONTEND

React 18 • TypeScript 5 • Vite 5 • TanStack Query • Tailwind

BACKEND

Spring Boot 3.3.4 • Java 21 • Spring Data JPA • Maven

SECURITY

Spring Security • JWT access/refresh • BCrypt • role checks

DATABASE

MySQL 8 • Hibernate / JPA • 17 entities

ML SERVICE

FastAPI 0.115 • scikit-learn 1.5.2 • pandas • joblib

TESTING

JUnit 5 • Mockito • 17 backend test files • JaCoCo

INTEGRATIONS

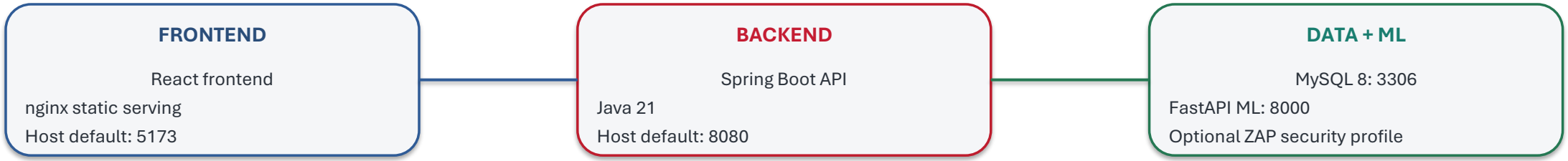
SonarQube • Trivy • OWASP ZAP • JaCoCo • JGit

INFRASTRUCTURE

Docker • Docker Compose • GitHub Actions • nginx

DEPLOYMENT

Containerized Deployment



Docker Compose starts four core services: MySQL, backend, ML service and frontend. The frontend nginx configuration reverse-proxies /api requests to the backend. A read-only repository mount supports JGit code-churn analysis.

Start with: docker compose up --build | Optional ZAP: docker compose --profile security up

IMPLEMENTATION STATUS

Implementation Evidence and Verified Scope

IMPLEMENTED MODULES

- JWT authentication, roles and project ownership.
- Project CRUD and source-file upload workflow.
- Rule-based quality and security analysis.
- JaCoCo, SonarQube, Trivy and ZAP integration paths.
- JGit-based code churn analysis.
- Hybrid rule-based + ML risk assessment.
- Adaptive test prioritization and execution history.
- MLOps endpoints, prediction logging and JSON reports.
- GitHub Actions CI/CD workflow and Docker Compose.

DISCLOSURES

- ML dataset is synthetic/sample data.
- Four ML features are project-level rather than per-file.
- Backend Maven tests were not run in the original build environment due to Maven Central network restrictions; run locally to confirm.

DEMONSTRATION

Final Review Demonstration Plan

CORE APPLICATION FLOW

1. Start the Docker Compose stack.
2. Sign in and create/select a project.
3. Upload source files.
4. Run quality and security analysis.
5. Review quality metrics and findings.
6. Run component risk analysis.
7. Create test cases and prioritize them.
8. Record execution outcomes.
9. Review dashboard, churn and history.
10. Generate the project JSON report.

INTEGRATION FLOW

If configured, demonstrate JaCoCo import, SonarQube metrics, Trivy scan/report import, ZAP baseline/report import and the MLOps model information / prediction history views.

Demonstrate optional integrations only when their service, credentials or report files are available.

LIMITATIONS + ROADMAP

Known Limitations and Next Steps

CURRENT LIMITATIONS

- ML training set is synthetic/sample data.
- Four ML features are shared at project level, not measured per file.
- No Flyway/Liquibase migration tool yet.
- Source Code tab is listed in navigation but has no registered frontend route/page.
- Trivy CLI requires installation in the backend environment; JSON import is supported.
- SonarQube and ZAP live modes require external services and configuration.
- Backend Maven tests/build require local verification.

FUTURE ENHANCEMENTS

- Train and evaluate with labelled historical project data.
- Add per-file quality and coverage metrics.
- Introduce Flyway/Liquibase schema migrations.
- Complete Source Code frontend page.
- Improve deployment secrets and observability.
- Expand CI/CD quality gates and remediation guidance.

These are disclosed constraints and planned improvements, not capabilities claimed as complete.

PROJECT REFERENCES

Project References and Technical Sources

1. ASSQP README.md — implemented scope, architecture, modules and known limitations.
2. ASSQP Project System Design PDF — submitted system-design architecture and flow.
3. ASSQP backend source — Spring Boot controllers, services, entities and tests.
4. ASSQP frontend source — React pages, API clients, routes and dashboard components.
5. ASSQP ML service — FastAPI endpoints, training pipeline and model metadata.
6. .github/workflows/ci-cd.yml — pipeline implementation.
7. docs/API_ENDPOINTS.md and Swagger / OpenAPI specification.

SOURCE BASIS • Uploaded project documentation and repository implementation

Thank You

Questions & Discussion

PROJECT TEAM

2420030540 | Allipuram Akhil
2420030546 | P. Ruthvik Reddy
2420030579 | S. Akshay Veer
2420030141 | B. Akhil Varma

Faculty / Guide: Dr. J. Kavitha Reddy